Interview cheat sheet: all lessons

Every lesson's cheat sheet in one place. ◆ = my addition beyond the video; everything else is from the lessons.

01 · What Is Inference & Inference Engineering?

Full notes

Say it in 30 seconds

> *Inference is running a trained model token by token for every request, so fleet-wide it costs more than training. Decode is memory-bandwidth-bound: every step streams all the weights to make one token per sequence. The core trade is latency vs throughput vs cost, and batching is lever one. The job: measure, find the bottleneck, change one thing, repeat.*

Only numbers worth memorizing

- **H100:** 3.35 TB/s HBM, ◆ ~989 TFLOPS BF16. **Llama 3 8B:** 16 GB in BF16, ~128 KB KV per token (taught in 02).
- **Software matters:** batching gives ~15× cheaper tokens; vLLM gets up to 24× naive HF throughput.

Derive, don't memorize

Let P = parameters, B = batch size, BW = memory bandwidth.

1. Decode speed = bandwidth ÷ bytes per step

Every decode step streams all the weights once, so time per step is bytes moved over bytes per second.

$$t_{\text{step}} \approx \frac{2P \text{ bytes}}{\text{BW}} = \frac{16 \text{ GB}}{3.35 \text{ TB/s}} \approx 4.8 \text{ ms} \quad \Rightarrow \quad \leq 200 \text{ tok/s per user}$$

> **INTUITION**
>
> Halve the bytes (8-bit weights) and the ceiling doubles. That's why precision is a speed lever.

2. Memory-bound or compute-bound? Compare FLOPs per byte to the ridge ◆

Each weight does a multiply and an add (2 FLOPs) per token and costs 2 bytes to read. One read is shared by all B sequences:

$$\frac{\text{FLOPs}}{\text{byte}} \approx \frac{2P \cdot B}{2P} = B \qquad \text{vs} \qquad \text{ridge} = \frac{989 \text{ TFLOPS}}{3.35 \text{ TB/s}} \approx 300$$

Batch 1 gives ~1, far below 300, so decode is memory-bound and batching is nearly free speed. Prefill puts ~500 prompt tokens through one read, above the ridge, so it's compute-bound:

$$t_{\text{prefill}} \approx \frac{N_{\text{prompt}} \cdot 2P}{\text{peak FLOPS}} = \frac{500 \times 16 \text{ GFLOP}}{989 \text{ TFLOPS}} \approx 8 \text{ ms}$$

3. Cost = what you pay per hour ÷ what you make per hour

$$\frac{\$}{1\text{M tokens}} = \frac{\$ \text{ per GPU hour}}{\text{tok/s} \times 3600} \times 10^6 \qquad \frac{3}{200 \times 3600} \times 10^6 \approx 4.17 \qquad \frac{3}{3000 \times 3600} \times 10^6 \approx 0.28$$

4. Why batching isn't free: every sequence brings its own KV cache ◆

A step reads the weights once plus every sequence's cache (context length C; KV/token ≈ 128 KB, derived in 05), so steps slow down as the batch grows:

$$t_{\text{step}}(B) \approx \frac{2P + B \cdot C \cdot \text{KV/token}}{\text{BW}}$$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch × context (KV reads)

Symptom → lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High $/token, GPU underused	Batch too small	Scheduling: batch more requests per step
Batch capped by out-of-memory	KV cache fills memory	Memory: pack KV tightly, reuse shared prefixes
Slow tokens even at low load	Too many weight bytes per step	Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	Shorter prompts, more replicas (routing fixes: see 02)
Model doesn't fit one GPU	Size	Parallelism across GPUs, or precision

Rapid-fire Q&A

Question	Crisp answer
Why does batching help, and what does it cost?	One weight read serves all *B* sequences. The cost is KV memory and slower steps (derivation 4).
◆ Max tok/s for 70B BF16 on 8×H100?	Derivation 1: 140 GB ÷ (8 × 3.35 TB/s) ≈ 5.2 ms/step, so ≤ ~190 tok/s before communication overhead.
Cost too high: where do you start?	Measure (utilization, batch size, TTFT/ITL) before changing anything; then check off-GPU wins such as a smaller model, shorter prompts, caching.

TRAP

- "Just call `generate()`" or "buy a faster GPU": both ignore the scheduler, memory and setup.
- Trading away quality: a faster but worse answer is not a win.

02 · The Inference Stack: Model to GPU to Production

Full notes

Say it in 30 seconds

> *A request crosses about eight layers. The model defines the math; kernels, CUDA and the GPU set how fast it runs, bounded by bytes ÷ bandwidth. The inference engine batches requests every step and manages KV cache memory. The API server, router and autoscaler pick the replica and the replica count. Nearly all time is GPU time; the upper layers decide how well it's used.*

Only numbers worth memorizing

- **H100:** 80 GB HBM, 132 SMs. **Llama 3.1 8B:** 32 layers. Routing ~1 ms; decode ~5 ms/token.

Derive, don't memorize

Every derivation here is the same idea: **time = bytes ÷ bandwidth**, or **capacity = bytes ÷ bytes-per-item**. Per-step time: see 01, derivation 1. KV per token: see 05, derivation 2.

1. KV capacity = memory left after the weights ÷ cache per token ◆

$$N_{\text{tokens}} \approx \frac{\text{HBM} - \text{weights}}{\text{KV/token}} = \frac{80 - 16\ \text{GB}}{128\ \text{KB}} \approx 488\text{k} \approx 119\ \text{requests} \times 4\text{k}$$

> **INTUITION**
>
> That token budget, not compute, caps the batch. It's why engines manage KV in blocks and why quantizing weights or KV frees room for more users.

2. Launch overhead = kernels × launch cost, against the step time ◆

$$\frac{N_{\text{kernels}} \times t_{\text{launch}}}{t_{\text{step}}} = \frac{300 \times 4\ \mu\text{s}}{4.8\ \text{ms}} \approx 25\%$$

> **INTUITION**
>
> Illustrative numbers, but the shape is real: at small batch, CPU launch time is a big slice of each step. Fusion (and ◆ CUDA graphs) cut it.

3. Cold start = weight bytes ÷ load bandwidth ◆

$$t_{\text{cold}} \approx \frac{16\ \text{GB}}{2\ \text{GB/s}} = 8\ \text{s}$$

> **INTUITION**
>
> Assumes ~2 GB/s from local disk; from network storage it's often slower. Scaling reacts late, so autoscalers must scale before the queue explodes.

Layer → question → what to watch

Layer	Answers	Tools	Watch / lever
Model + framework	What to compute	Weights, recipe, tokenizer, PyTorch	Model size, precision
Kernels + CUDA + GPU	How fast the math runs	cuBLAS, FlashAttention, CUDA	Bytes moved, fusion, launches
Inference engine	Who runs each step; where memory goes	vLLM, SGLang, TensorRT-LLM	Batch size, KV usage
API server	Speaks HTTP, tokenizes, streams	OpenAI-compatible server	Streaming latency
Router + autoscaler	Which replica; how many	Prefix-aware router	Queue length, cache hits

Symptom → layer to investigate

Symptom	Layer	First fix
GPU idle between kernels, CPU busy	Kernels	Fuse kernels; cut launches
Same math slower than expected	Kernels	Better kernels (cuBLAS, FlashAttention)
Repeated system prompts inflate TTFT	Router	Prefix-aware routing
Queue grows while CPU looks idle	Autoscaler	Scale on queue length and KV use
New replicas arrive too late	Fleet	Derivation 3: scale earlier, load faster

Rapid-fire Q&A

Question	Crisp answer
What does an inference engine add over PyTorch?	A per-step scheduler (batching) and a KV cache memory manager.
Why not autoscale on CPU?	The work is on the GPU; the CPU idles. Watch queue length and KV use.
Why is a round-robin load balancer wasteful?	It scatters shared prompts, so each replica re-prefills them.
What's a replica?	One engine on its GPU(s), behind an API server.
Where does a request's time go?	Mostly GPU (prefill, then ~5 ms per decode step); upper layers decide utilization.

TRAP

- Treating LLM serving like a stateless web app: round-robin routing and CPU-based autoscaling.
- Assuming the model file determines speed: it only defines the math.

03 · Levels of Inference Engineering

Full notes

Say it in 30 seconds

> *Inference work sits on four levels: the user picks and calls models, the operator makes a model fit and stay up, the optimizer gets more tokens per GPU within latency targets, and the engine builder changes the engine itself. Each level is easier if you understand the one below. The key skill is finding which level a problem lives on; cheap fixes on lower levels often beat kernel work.*

Only numbers worth memorizing

- **DeepSeek-R1 (Feb 2025):** $0.55/M input, $2.19/M output (~4×), $0.14/M cached input.
- **Batching measured:** MPT-7B/A100 0.9 → 12.5 req/s (~14×).

Derive, don't memorize

1. API bill = tokens × price, per token type

Each token type has its own price, and a cached prefix bills at the cached rate:

$$\text{cost} = N_{\text{cached}} \cdot p_{\text{cached}} + N_{\text{in}} \cdot p_{\text{in}} + N_{\text{out}} \cdot p_{\text{out}} \quad \Rightarrow \quad \$2.42 \to \$1.19 \text{ per 1k requests}$$

> **INTUITION**
>
> Put stable text first (caches match prefixes ◆, so any change early in the prompt invalidates everything after it), and cap output length: output tokens cost ~4×.

The operator's fit check (params × bytes per param vs GPU memory): see 04, derivation 1.

The four levels

Level	Question	Controls	Example win
1 · User	Which model, called how?	Model, prompt, output length, streaming, caching	Stable prefix first: bill ÷ 2
2 · Operator	Does it fit and stay up?	GPUs, precision, startup, failover, rollouts	Day/night fleet sizing
3 · Optimizer	More per GPU within latency?	Batching, quantization, prefix caching, speculative decoding, PD split	~14× from batching
4 · Engine builder	What should the engine do differently?	Schedulers, kernels, memory layout	Orca, FlashAttention, PagedAttention

Symptom → which level

Symptom	Level	First fix
API bill high, same instructions every request	1	Identical block at the front for cache hits
Model won't load on one GPU	2	Split across GPUs or quantize
Fits at launch, out of memory at peak	2	Budget KV for peak concurrency
Attention kernel dominates the profile	4	Kernel work (after levels 1–3 are exhausted)

Rapid-fire Q&A

Question	Crisp answer
Why do output tokens cost ~4× input?	Decode is serial and memory-bound; prefill is parallel (see 01, derivation 2).
Why split prefill and decode onto separate machines?	◆ Prefill is compute-bound, decode memory-bound; each group is tuned for its own job.
What did Orca, FlashAttention, PagedAttention contribute?	Per-step scheduling (continuous batching); no big attention matrix in HBM; paged KV cache (vLLM).
Is the engine builder the "senior" level?	No: levels aren't seniority, and most savings come from levels 1–3.

TRAP

- Jumping to level 4 (kernels) before checking model choice, prompts and caching.
- Sizing memory for launch traffic instead of peak.

04 · Math for Memory, Throughput & Speedups

Full notes

Say it in 30 seconds

> *Every number is an amount, a rate or a ratio, and time = amount ÷ rate. Weight memory is params × bytes per param, so precision is the first lever for fit. Divide bytes by bandwidth and FLOPs by FLOPS to get floors; real runs land above them. Latency and throughput are reciprocals only for one stream. Speedups are before ÷ after, and Amdahl's law caps them by the fraction of time touched.*

Only numbers worth memorizing

- **Bytes per param:** FP32 4, BF16 2, FP8 1, INT4 0.5.

Byte units: decimal vs binary

Decimal (SI)	Bytes	Binary (IEC)	Bytes	Binary ÷ decimal
1 KB (kilobyte)	10^3 = 1,000	1 KiB (kibibyte)	2^{10} = 1,024	1.024
1 MB (megabyte)	10^6	1 MiB (mebibyte)	$2^{20} = 1024^2 \approx 1.049 \times 10^6$	1.049
1 GB (gigabyte)	10^9	1 GiB (gibibyte)	$2^{30} = 1024^3 \approx 1.074 \times 10^9$	1.074
1 TB (terabyte)	10^{12}	1 TiB (tebibyte)	$2^{40} = 1024^4 \approx 1.100 \times 10^{12}$	1.100

The "i" means binary: kibi = **ki**lo **bi**nary, mebi = **me**ga **bi**nary, and so on. Say "kibibyte" (KIB-ee-byte), "gibibyte" (GIB-ee-byte).

- **GB → GiB:** divide by 1.074. 16 GB = **14.9 GiB**; an 80 GB H100 = **74.5 GiB**.
- ◆ Datasheet rates are decimal: **1 TB/s = 1 GB/ms = 1 MB/μs**, so 3.35 TB/s moves 16 GB in 16 ÷ 3.35 ≈ 4.8 ms.

Derive, don't memorize

1. Weights = params × bytes per param; check fit first

$$\text{weights} = N \times \frac{\text{bits}}{8}: \quad 70\text{B} \to 140\ (\text{BF16}),\ 70\ (\text{FP8}),\ 35\ (\text{INT4})\ \text{GB vs } 80\ \text{GB}$$

> **INTUITION**
>
> "Fits" means weights + KV cache at peak + runtime (KV capacity: see 02, derivation 1). Too big: split across GPUs or quantize.

2. Time = amount ÷ rate, and the slower floor wins ◆

Each token must both move bytes and do math; whichever takes longer sets the floor:

$$t \geq \max\left(\frac{\text{bytes}}{\text{BW}}, \frac{\text{FLOPs}}{\text{FLOPS}}\right) \quad 70\text{B FP8}: \frac{70}{3350} \approx 21\ \text{ms} \qquad \text{INT4}: \approx 10\ \text{ms}$$

INTUITION

A measurement below the floor means an assumption broke (fewer bytes moved, or a bad measurement). Why bytes usually win: see 01, derivation 2.

3. Speedup = before ÷ after; Amdahl caps it

$$S = \frac{t_{\text{before}}}{t_{\text{after}}} \qquad S_{\text{overall}} = \frac{1}{(1-p) + p/s} \leq \frac{1}{1-p}$$

INTUITION

"x% faster" = 1 + x; "x% less time" = 1 / (1 − x). 30% of a step made 3× faster gives 1.25×; infinitely faster gives 1.43×.

Symptom → diagnosis

Symptom	Diagnosis
Tool shows 14.9 for a 16 GB file	GiB vs GB, not a bug
Measured faster than the bandwidth floor	Fewer bytes moved than assumed, or a broken measurement
Kernel 3× faster, end to end barely moves	Small p: Amdahl
"1,800 tok/s" but users say it's slow	Throughput across many streams; ask per-stream speed

Rapid-fire Q&A

Question	Crisp answer
FLOPs vs FLOPS?	Count (what a model costs) vs per second (what a GPU supplies).
Why are napkin numbers floors?	Datasheet peaks are never sustained, so real runs are slower.
20 ms → 15 ms: how much faster?	1.33× (25% less time), not 1.25×.
Is a throughput gain a latency gain?	Only for one stream; with batching they decouple.

TRAP

- Adding speedups ("2× + 2× = 4×"): compose by time accounting instead.
- Quoting a speed floor for a model that doesn't fit.

05 · Dot Products, Softmax & Attention Math

Full notes

Say it in 30 seconds

> *Attention scores are dot products of the new token's query with every past key, scaled by √d and softmaxed into weights that mix the values. Per decode token that's little arithmetic but a full read of every stored key and value, about 4 ops per byte, so decode attention is memory-bound. K/V grows 128 KB per token per user in Llama 3 8B, overtaking the weights at long context or big batch. Prefill instead faces an n² score matrix, which FlashAttention avoids with an online softmax.*

Only numbers worth memorizing

- **Llama 3 8B attention:** 32 layers, 32 query heads, 8 KV heads (GQA), head dim 128. **FP16 max:** 65,504.

Derive, don't memorize

Let L = layers, H = heads, d = head dim, n = context length.

1. Attention ops per decode token = dots × cost per dot × 2 (scores + weighted sum)

$$\text{ops} \approx L \cdot H_q \cdot n \cdot 2d \cdot 2 = 32 \cdot 32 \cdot 4096 \cdot 256 \cdot 2 \approx 2.1\ \text{B} \quad (\text{vs} \approx 15\ \text{B for weights})$$

INTUITION

The final ×2 = scores + weighted sum of values. Where the weights' ~15B comes from: see 07, derivation 2.

2. KV bytes per token: one K and one V per KV head per layer

$$\text{KV/token} = 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot \text{bytes} = 2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 128\ \text{KB} \quad \Rightarrow \quad \frac{2.1\ \text{B ops}}{4096 \times 128\ \text{KB}} \approx 4\ \text{ops/byte}$$

INTUITION

4 ≪ ~300 ridge (01, derivation 2): memory-bound. K/V matches the 16 GB weights at 16 GB ÷ 128 KB ≈ 122k tokens for one user, or ~4k each for 32 users. GQA's 8 KV heads instead of 32 cut this 4× (Llama 2 7B: 512 KB/token, see 07).

3. Prefill scores = n² per head

$$n^2 \cdot H \cdot 2\ \text{B} = 8192^2 \times 32 \times 2 \approx 4.3\ \text{GB per layer}$$

INTUITION

Quadratic scratch is why FlashAttention tiles keys and never writes the square to memory.

4. Online softmax: running max *m* and sum *l*, rescaled when *m* rises

$$m' = \max(m, m_{\text{blk}}) \qquad l' = l \cdot e^{m - m'} + \sum_{\text{blk}} e^{x - m'}$$

INTUITION

Blocks [2, 1] then [3, 0]: $l = 1.37 \rightarrow 1.37 \cdot e^{-1} + e^{0} + e^{-3} \approx 1.55$, identical to the full row.

Decode attention vs prefill attention

	Decode (1 new token)	Prefill (n prompt tokens)
Queries	1 per head	n per head
Problem	Reading all K/V (bytes)	n × n score matrix (scratch)
Grows with	Context × batch	Prompt length squared
Main fixes	GQA, KV quantization ◆, paged KV	FlashAttention (tiling + online softmax)

Rapid-fire Q&A

Question	Crisp answer
Why divide by √d?	Random q·k has variance d; unscaled scores make softmax one-hot.
Why subtract the max in softmax?	FP16 overflows ($e^{12} > 65{,}504$); the weights are unchanged.
What does GQA save?	K/V memory and bandwidth: 8 shared KV heads instead of 32.
Is attention the bottleneck?	Only at long context or big batch; otherwise the weights dominate.

TRAP

- Counting attention FLOPs instead of K/V bytes for decode.
- Calling softmax "cheap": its whole-row dependency dictates kernel design.

06 · Probability, Sampling & Benchmark Statistics

Full notes

Say it in 30 seconds

> *Each generated token is a random draw from the model's distribution, so reply text and length vary and latency becomes a long-tailed distribution. Report P50, P95 and P99, not the mean, and state the sample count: you need about 100 samples beyond a percentile for ~10% precision. Many calls per task amplify the tail. Treat a speedup as real only if it exceeds run-to-run noise, measured with the standard error σ/√n.*

Only numbers worth memorizing

- **Samples for ±10%:** P50 → 200, P95 → 2,000, P99 → 10,000 requests. **10 calls at P99:** ~9.6% of tasks hit the tail.

Derive, don't memorize

1. Tail amplification: chance at least one of *k* calls lands in the slowest fraction *q*

$$P = 1 - (1-q)^k = 1 - 0.99^{10} \approx 9.6\%$$

INTUITION

For small q, P ≈ $k{\cdot}q$ ◆. An agent making 50 calls hits a P99-slow call ~40% of the time: set per-call targets at a high percentile.

2. Samples needed: ~100 beyond the percentile, error ≈ 1/√100

$$n \approx \frac{100}{1-p} \qquad \text{P99}: \frac{100}{0.01} = 10000 \qquad \text{error} \approx \frac{1}{\sqrt{100}} = 10\%$$

3. Noise band: standard error shrinks with √n

$$\text{SE} = \frac{\sigma}{\sqrt{n}} = \frac{0.8}{\sqrt{5}} \approx 0.4 \qquad \text{band}_{\text{diff}} \approx 2 \times \sqrt{2} \times \text{SE} \approx \pm 1.0$$

INTUITION

◆ √2 because both averages are noisy; ×2 for ~95% confidence. 4× the runs halves the band. 0.7 < 1.0: not proven.

Mean vs median vs percentiles

	Mean	Median (P50)	P95 / P99
Tells you	Average cost (capacity math) ◆	The typical request	The tail users complain about
Long tail effect	Dragged right (1.3 vs 1.1 s)	Unaffected	Is the tail
Samples needed	Few	~200	~2,000 / ~10,000

Symptom → diagnosis

Symptom	Diagnosis
Mean latency fine, users complain	Long tail: look at P95/P99
P99 jumps between identical reruns	Too few samples beyond P99
"2% faster" from one run each	Compare to run-to-run noise first
Same prompt, very different request times	Sampled reply lengths differ

Rapid-fire Q&A

Question	Crisp answer
Why does the same prompt give different outputs?	Each token is a random draw from a distribution.
Why are SLOs written as percentiles?	The tail is what users feel, and multi-call tasks amplify it.
How do you know a speedup is real?	Repeat runs; the difference must exceed the noise band.
◆ How do temperature / top-p relate?	They reshape or truncate the distribution before the draw.

TRAP

- Reporting only the mean for a long-tailed latency distribution.
- Quoting P99 from a few hundred requests without saying so.

07 · Transformer Architecture Refresher

Full notes

Say it in 30 seconds

> *A decoder transformer is an embedding lookup, a stack of identical blocks with attention and an MLP adding onto a residual stream, and an LM head over the vocabulary. From the config you can count parameters (Llama 3 8B: 8.03B, 70% in MLPs), turn them into bytes with the precision, and estimate ~2 ops per non-embedding parameter per token. The KV cache, set by the number of KV heads, is the line that grows. MoE, GQA and RoPE each move one line of that bill.*

Only numbers worth memorizing

- **Llama 3 8B config:** hidden 4,096 · MLP 14,336 · 32 layers · 32 heads · 8 KV heads · vocab 128,256 · bf16.
- **Shares:** MLP 70%, attention 17%, embedding + head 13%. **Mixtral 8x7B:** 46.7B stored, 12.9B active.

Derive, don't memorize

Let h = hidden, i = MLP size, V = vocab, L = layers, $H_{kv}{\cdot}d$ = KV width ($8 \times 128 = 1{,}024$).

1. Parameters from the config

$$\text{block} = 2h^2 + 2h \cdot H_{kv}d + 3hi \qquad N = L \cdot \text{block} + 2Vh = 32 \times 218\text{M} + 2 \times 525\text{M} \approx 8.03\text{B}$$

The three terms are Q and O ($h \times h$), K and V ($h \times H_{kv}d$), and the three MLP matrices ($h \times i$).

> **INTUITION**
>
> The $2Vh$ assumes an untied LM head (Llama 3 8B). If embeddings are tied ◆, count Vh once.

2. Ops per token = 2 × non-embedding parameters

$$2 \times (8.03\text{B} - 0.53\text{B}) \approx 15\text{B ops/token}$$

$$\text{prefill 500 tokens} = 7.5\text{T ops per weight read} \qquad \text{decode} \approx \frac{15\text{B ops}}{16\text{ GB}} \approx 1\text{ op/byte}$$

3. MoE splits memory from compute

$$\text{memory} \propto N_{\text{total}} = 46.7\text{B} \qquad \text{ops/token} \approx 2N_{\text{active}} = 2 \times 12.9\text{B}$$

> **INTUITION**
>
> ◆ In BF16 that's ~93 GB to host but only ~26B ops per token. KV per token comes straight from the config too: see 05, derivation 2.

Swap → which line of the bill

Swap	Weight memory	Ops per token	KV cache	Context
Mixture of experts	↑ all experts	↓ only active		
Fewer KV heads / compressed K,V			↓	
RoPE positions				↑ (Llama 3.1: 128k)
Lower precision	↓ bytes			

Rapid-fire Q&A

Question	Crisp answer
Where do most parameters live?	MLPs (~70%); > 4× attention per block.
Does 8B params mean 16 GB is enough?	No: weights are the floor; KV cache and working memory add on top.
Which parameters cost no compute?	The embedding lookup, and idle experts in MoE.
Why did Llama 3 use 8 KV heads?	GQA cuts the cache 4× vs Llama 2 7B (128 vs 512 KB/token).
Why quantize MLPs first?	They hold most bytes, and decode time ∝ bytes read.

TRAP

- Equating parameter count with required memory.
- Assuming MoE is cheap to host because it's cheap per token.

08 · Generate & Stream Your First Tokens

Full notes

Say it in 30 seconds

> *A chat request is templated into a token sequence ending in an assistant header, the generation prompt. generate() then loops one forward pass per token: the first pass prefills the whole prompt and caches K/V (the pause, TTFT); each later pass reads all weights for one token (the pace, TPOT). Streaming is a put() hook on that loop, holding back partial words. It stops on an EOS token, a stop string or max_new_tokens. Measure tokens/s from generated tokens only, with the GPU synchronized.*

Only numbers worth memorizing

- **Llama 3.1 defaults:** 3 EOS ids, do_sample on, temperature 0.6, top-p 0.9. **Transformers `max_new_tokens`:** 20 if unset.

Derive, don't memorize

1. Request time = the pause + the pace × remaining tokens ◆

$$t_{\text{request}} \approx \text{TTFT} + (N_{\text{out}} - 1) \cdot \text{TPOT} \approx 0.31\ \text{s} + 59 \times 5.3\ \text{ms} \approx 0.62\ \text{s}$$

> **INTUITION**
>
> TTFT grows with prompt length (prefill); TPOT's floor is weight bytes ÷ bandwidth (01, derivation 1). Long prompts hurt the pause; big models hurt the pace.

2. Reported tokens/s = generated tokens ÷ time since the first token

$$\frac{60}{0.316\ \text{s}} \approx 190\ \text{tok/s} \qquad \text{counting the prompt}: \frac{90}{0.316} \approx 285\ (1.5\times\ \text{too high})$$

> **INTUITION**
>
> Without a GPU sync the timer stops early and the rate looks absurdly high: synchronize or use CUDA events.

Streamer options

	TextStreamer	TextIteratorStreamer
Output	Prints to stdout	Pushes text into a queue
Use	Notebooks, CLI	Servers: read without blocking the loop
Shared behavior	`put()` per token; holds back after the last space; flushes on newline	Same

Symptom → diagnosis

Symptom	Diagnosis
Answer cut off mid-sentence	`max_new_tokens` unset (default 20)
Model continues writing the user's question	Missing generation prompt (◆ `add_generation_prompt=True`)
Stream lags the model by a word	Hold-back by design, not a bug
tokens/s implausibly high	Counted the prompt, or no GPU sync
Same prompt, different answers	Sampling is on by default (temperature 0.6, top-p 0.9)

Rapid-fire Q&A

Question	Crisp answer
What's the generation prompt?	The trailing assistant header that cues the reply.
Why is the first token slow?	It runs prefill over the whole prompt and fills the KV cache.
Does streaming speed up generation?	No: it changes when you see tokens, not when they're made.
Why hold back text after the last space?	The next token can change how a partial word decodes.

TRAP

- Timing `generate()` with a plain stopwatch and no GPU sync.
- Leaving `max_new_tokens` unset in production.

09 · Watch GPU Memory, Utilization & Power

Full notes

Say it in 30 seconds

> *Dashboard memory is weights plus the allocator's cached pool plus the KV cache plus a driver reserve, and engines like vLLM pre-book 92% at launch for KV blocks. GPU utilization is a duty cycle, the share of time any kernel ran, so one user's decode can read 100% while compute idles. Power below the cap is normal for memory-bound decode; caps and thermal events show up in clock reasons. For how well the GPU is used, measure MBU and MFU against a floor.*

Only numbers worth memorizing

- **vLLM** `gpu_memory_utilization` default **0.92**. **Util window:** 1/6 s–1 s. **H100 SXM:** up to 700 W. **P0** full … **P12** idle.

Derive, don't memorize

1. Dashboard memory is a sum, nested around your tensors

$$\text{used} = W + \text{pool} + \text{KV} + \text{reserve} \qquad \text{allocated} \le \text{reserved} \le \text{used}$$

INTUITION

Engine pre-book: 0.92 × 80 ≈ 73.6 GB, so ≈ 57.6 GB above the 16 GB weights becomes KV blocks ◆ (minus activations). How many tokens that holds: see 02, derivation 1.

2. Utilization = time any kernel ran ÷ sample window

$$\text{util} = \frac{t_{\text{any kernel}}}{t_{\text{window}}} \qquad 1 \text{ of } 132 \text{ SMs busy all window} \Rightarrow 100\%$$

3. "Busy enough" = achieved ÷ peak, for bytes and for ops ◆

$$\text{MBU} = \frac{16 \text{ GB}/6 \text{ ms}}{3.35 \text{ TB/s}} \approx 80\% \qquad \text{MFU} = \frac{15 \text{ GFLOP}/6 \text{ ms}}{989 \text{ TFLOPS}} \approx 0.25\%$$

INTUITION

Decode: judge by MBU (bandwidth-bound). Prefill and big batches: MFU matters too (01, derivation 2).

What each reading shows

Reading	Shows	Doesn't show	Check instead
Memory used	What the driver handed out	Live tensors alone	`memory_allocated()`
GPU-util	Any kernel running	SMs busy, bandwidth used	MBU / MFU
Memory-util	Memory bus active time	Bytes moved	MBU
Power draw	What the board spends	How hard it works	Clocks event reasons

Symptom → diagnosis

Symptom	Diagnosis
~92% memory before any request	Engine pre-booked KV blocks, not a leak
Memory climbs during long chats	KV cache growth
Growth survives finished requests + `empty_cache()`	A real leak
100% util with one user	Duty cycle; compute mostly waiting on memory
Throughput dips, traffic unchanged	Power cap, thermal slowdown or high P-state

Rapid-fire Q&A

Question	Crisp answer
Why is "used" bigger than my tensors?	Allocator pool, KV cache and driver reserve are included.
Does 100% util mean saturated?	No: it means a kernel was running.
Decode at half the power cap: problem?	No: memory-bound decode leaves compute idle.
Where do power or heat throttles show?	Clocks event reasons (SW Power Cap, thermal slowdown).

TRAP

- Treating GPU-util as saturation, or power draw as effort.
- Calling an engine's launch-time pre-booking a memory leak.

10 · Trace the Transformer Forward Pass

Full notes

Say it in 30 seconds

> *One forward pass per token: the tokenizer (CPU, microseconds) turns text into IDs, each ID picks an embedding row to form a (batch, positions, 4,096) residual stream, 32 blocks each read the stream and add an adjustment without changing its shape, and the final norm plus output matrix produce logits of shape (batch, positions, 128,256). The model scores every position; generation reads only the last.*

Only numbers worth memorizing

- **"The capital of France is":** 6 IDs → stream (1, 6, 4,096) → logits (1, 6, 128,256); **33** hidden states (embedding + 32 blocks).

Derive, don't memorize

Let B = batch, T = positions, h = 4,096, V = 128,256, L = 32.

1. Shapes through the pass

$$(B, T) \to (B, T, h) \to \cdots \to (B, T, h) \to (B, T, V) \qquad \text{hidden states} = L + 1 = 33$$

2. Full-sequence logits are big ◆

$$B \cdot T \cdot V \cdot 4\ \text{B} = 8192 \times 128256 \times 4 \approx 4.2\ \text{GB} \quad \text{vs} \quad 128256 \times 4 \approx 0.5\ \text{MB for the last row}$$

> **INTUITION**
>
> ◆ Serving only needs the last position's logits, so engines project just that row. Kept hidden states cost (L+1)·T·h·2 B ≈ 2.2 GB at 8k: debugging only.

hidden_states index map

Index	What it is
0	Embedding output, before any block
1 … 31	Stream after block 1 … 31
32	After block 32 **and** the final norm

Symptom → diagnosis

Symptom	Diagnosis
`hidden_states[-1]` ≠ what you expect before the head	It's already final-normed
Wrong next token from your own loop	Read `logits[:, 0]` instead of `logits[:, -1]`
◆ OOM on long prompts in a plain forward	Full (B, T, V) logits or kept hidden states

Rapid-fire Q&A

Question	Crisp answer
What's the last non-neural step?	The tokenizer: lookup + merge rules on the CPU.
What does an embedding lookup do?	Each ID selects one row; no multiplication.
Does a block change the stream's shape?	No: it reads, computes an adjustment, adds it back.
Why are there 33 hidden states for 32 blocks?	Index 0 is the embedding output.

TRAP

- Assuming the model scores only the last position: it scores all of them.
- Treating `hidden_states[32]` as the raw block output: the final norm is applied.

11 · Q, K, V & Causal Attention During Inference

Full notes

Say it in 30 seconds

> *Each token's hidden state is projected into a query, key and value. Scores are q·k / √d, softmaxed into weights, and the output is the weighted sum of values. A causal mask sets future scores to −∞, so the grid is a lower triangle and appending a token never changes earlier rows. Prefill computes the whole triangle; each decode step adds one row. Queries are used once, but keys and values are reread by every later token, so we cache them, and reading that cache makes decode memory-bound.*

Only numbers worth memorizing

- **Worked example:** q_on = (1, 2) → weights 4 / 29 / 59 / 8% → output **(0.67, 0.88)**, mostly "sat".

Derive, don't memorize

1. Masked attention in one line

Scores, mask, softmax, blend. The mask is added, not multiplied, so future cells become $\exp(-\infty) = 0$.

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right)V \qquad M_{ij} = 0 \ (j \le i), \quad -\infty \ (j > i)$$

INTUITION

Why $-\infty$ and not 0? A score of 0 still gets weight $e^0 = 1$. Only $-\infty$ gives exactly zero.

2. Work per step: the triangle vs one row

$$\text{prefill cells} = \frac{n(n+1)}{2} \qquad \text{decode step } t = t \text{ cells (one new row)}$$

INTUITION

Earlier rows can't change, so decode never redoes them: it only needs the new query and every past K and V. Bytes for those are in 05, derivation 2; the full recompute count without a cache is in 14, derivation 1.

Query vs key vs value

	Query	Key	Value
Role	What I'm looking for	What I offer for matching	What I hand over
Library	Your search	Spine labels	The books
Heads (Llama 3 8B)	32 × 128	8 × 128	8 × 128
Lifetime	Used once, at its own step	Reread by every later token	Reread by every later token
Cached?	No	Yes	Yes

Symptom → diagnosis

Symptom	Diagnosis
Training loss suspiciously low	Mask missing: the model peeks at future tokens
Earlier outputs shift when you append a token	◆ Mask bug: attention isn't causal
Decode slows as the conversation grows	Each step rereads all past K/V

Rapid-fire Q&A

Question	Crisp answer
Why separate keys and values?	Keys decide who is attended to; values decide what is passed on.
Is the mask needed at inference?	◆ Yes in prefill: prompt tokens run in parallel and must not see later prompt tokens.
What does a new token add to the grid?	One row: 1 query × all keys so far.
What's the shape of the score grid per head?	n × n queries by keys, lower triangle after masking.

TRAP

- Thinking the mask is training-only: prefill still needs it ◆.
- Caching queries: they're never reused.
- Multiplying by a 0/1 mask after softmax: rows no longer sum to 1. Add $-\infty$ before softmax.

12 · MLP, RMSNorm & Residual Connections

Full notes

Say it in 30 seconds

> *A transformer layer is two pre-norm sublayers on a residual stream: $x \leftarrow x + \text{attention}(\text{norm}(x))$, then $x \leftarrow x + \text{MLP}(\text{norm}(x))$. The stream carries 64 additive edits on top of the embedding, so early information survives and each sublayer only learns an edit. RMSNorm divides the copy by its root mean square and applies a learned gain, without mean subtraction. The MLP is per token: SiLU(gate) ⊙ up at 14,336 wide, then down to 4,096. It holds about 70% of the weights, so about 70% of decode's memory traffic. Norms and adds are memory-bound tiny ops, so they get fused.*

Only numbers worth memorizing

- **Llama 3 8B MLP:** 4,096 → **14,336** → 4,096; **81%** of a layer, **70%** of all weights (5.64B / 8.03B).

Derive, don't memorize

1. The stream is a sum of edits

Two adds per layer, nothing overwritten.

$$x_{\text{final}} = x_{\text{embed}} + \sum_{i=1}^{2L} \Delta_i \qquad 2L = 64 \text{ for } L = 32$$

2. RMSNorm by hand

Square, average, root, divide, then scale by the gain γ.

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{h}\sum_j x_j^2}} \qquad (4, -2, 2, -1) : \sqrt{\frac{25}{4}} = 2.5 \Rightarrow (1.6, -0.8, 0.8, -0.4)$$

INTUITION

LayerNorm adds two steps: subtract the mean first, add a learned β after. RMSNorm drops both. ◆ Real kernels add a tiny ε under the root to avoid dividing by zero.

3. The gated MLP

Two expansions, one gated by SiLU, multiplied, then shrunk.

$$\text{MLP}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x) \qquad \text{SiLU}(z) = z \cdot \sigma(z) \qquad \text{SiLU}(2) = 2 \times 0.88 \approx 1.76$$

4. The MLP's share of a decode step ◆

$$\frac{32 \times 176\text{M} \times 2\ \text{B}}{3.35\ \text{TB/s}} = \frac{11.3\ \text{GB}}{3.35\ \text{TB/s}} \approx 3.4\ \text{ms of the} \approx 4.5\ \text{ms step}$$

INTUITION

◆ By Amdahl (04), making only attention weights free caps decode speedup near 1.2×. Shrinking MLP weights (quantization, MoE) is the bigger lever.

Attention vs MLP vs norm + add

	Attention	MLP	RMSNorm + add
Mixes tokens?	Yes, across positions	No, per token	No, per number
Weights per layer	≈ 42M	≈ 176M	4,096 gains
Decode cost	Weights + KV reads	≈ 70% of weight reads	Memory passes, launches
Inference fix	GQA, KV cache	◆ Quantize, MoE	Fuse into one kernel

Symptom → diagnosis

Symptom	Diagnosis
Hidden-state values grow layer by layer	Normal: the raw stream accumulates edits; only copies are normalized
Profiler shows many tiny norm/add kernels	Unfused: each re-reads and re-writes the hidden state
◆ Quantized attention only, decode barely faster	MLP weights dominate the bytes read

Rapid-fire Q&A

Question	Crisp answer
Why residual connections?	Layers learn edits, and early information is never overwritten.
What's the difference between RMSNorm and LayerNorm?	No mean subtraction and no shift; works just as well.
Does the MLP see other tokens?	No. Attention moves information between tokens; the MLP is per token.
What does the gate do?	Picks which of 14,336 features pass for this token, and how strongly.
Why fuse add and norm?	Tiny math, but each would read and write the whole hidden state.

TRAP

- Saying the norm changes the stream: only the sublayer's copy is normalized.
- Assuming attention dominates the weights: the MLP is 81% of a layer.
- Judging norm/add cost by FLOPs: it's memory passes and launches.

13 · Logits, Sampling & the Next Token

Full notes

Say it in 30 seconds

> *The final-normed vector is dotted with the LM head's 128,256 rows to give logits, one score per token. Softmax exponentiates and normalizes them, and only logit differences matter. Greedy takes the argmax: repeatable, good for facts and code, but it can loop. Sampling divides logits by a temperature, cuts the tail with top-k, top-p or min-p, rescales, and draws; T → 0 is greedy. An engine applies each request's settings to its own logits row on the GPU in one pass, checks stop conditions, streams, appends, and loops. Fix the seed to reproduce samples.*

Only numbers worth memorizing

- **Toy logits 5 / 3.2 / 2.9 / 2.1 / 0.5** → 74 / 12 / 9 / 4 / <1%. Paris at T = 0.5: **96%**; at T = 1.5: **57%**.

Derive, don't memorize

1. Only gaps over T matter

Divide any two probabilities: the softmax denominator cancels, leaving the logit gap scaled by T.

$$\frac{p_i}{p_j} = e^{(z_i - z_j)/T} \qquad \frac{p_{\text{Paris}}}{p_{\text{the}}} = e^{1.8} \approx 6 \ (T=1), \quad e^{3.6} \approx 37 \ (T=0.5)$$

INTUITION

Add c to every logit and the gap is unchanged, so nothing moves. As T → 0 every ratio blows up and the top token takes everything: greedy. Temperature never reorders tokens.

2. Cut, then rescale

Keep a set S of tokens, then divide each survivor by the kept mass.

$$p'_i = \frac{p_i}{\sum_{j \in S} p_j} \qquad \text{top } p = 0.9 : 74, 86, 95 \Rightarrow 3 \text{ kept}, \quad p'_{\text{Paris}} = \frac{74}{95} \approx 78\%$$

INTUITION

Min-p 0.1 keeps p ≥ 0.1 × 74% = 7.4%: the same three tokens here. Top-k = 3 keeps them by count.

3. The head is a fixed cost per step ◆

$$V \cdot h \cdot 2 \text{ B} = 128256 \times 4096 \times 2 \approx 1.05 \text{ GB} \qquad \frac{1.05 \text{ GB}}{3.35 \text{ TB/s}} \approx 0.31 \text{ ms per decode step}$$

Decoding rules compared

Rule	What it does	Adapts to confidence?	Toy result
Greedy (T = 0)	argmax	n/a	Paris, always
Temperature	z / T before softmax	Reshapes, keeps order	Paris 96% (0.5), 57% (1.5)
Top-k	Keep the k most likely	No: fixed count	k = 3 → 78 / 13 / 10
Top-p	Smallest set with Σ ≥ p	Yes: nucleus grows when unsure	p = 0.9 → 3 tokens
Min-p	Keep p ≥ m × p_max	Yes: relative to the top	m = 0.1 → cutoff 7.4%

Symptom → diagnosis

Symptom	Diagnosis
Output repeats the same phrase	Greedy on open-ended text: sample, or add penalties
Rare bizarre token mid-answer	No tail cut: add top-p or min-p
Can't reproduce a sampled answer	Seed not fixed

Rapid-fire Q&A

Question	Crisp answer
What is a logit?	The dot product of the final vector with one LM-head row: an unnormalized score.
Why compute logits only at the last position?	Only it picks the next token; prefill runs the head for one position.
Why is temperature 0 allowed?	APIs treat it as greedy, the T → 0 limit.
Why top-p over top-k?	A fixed k ignores confidence; top-p's nucleus shrinks or grows.
What order does a sampler apply?	Penalties, temperature, cut the tail, rescale, draw.
How do mixed settings share a batch?	Each request's settings apply to its own logits row, in one GPU pass.
When does a request stop?	End-of-text token, maximum length, or a stop string.

TRAP

- Thinking temperature reorders tokens: it only sharpens or flattens.
- Forgetting to rescale after cutting the tail: survivors must sum to 1.
- Treating logits as probabilities: they can be negative and don't sum to 1.

14 · Why Autoregressive Generation Recomputes Work

Full notes

Say it in 30 seconds

> *The naive loop reruns the model over the whole sequence each step but uses only the last position's logits. Under the causal mask, old positions produce identical numbers every step, so all of that is recomputation, and it totals NP + N²/2 instead of P + N: 213× for 512 + 256. The new token only needs its own q, k, v and every old token's keys and values per layer, so we cache K and V. The KV cache is exact, costs ~128 KB per token on Llama 3 8B, and splits generation into one big prefill pass and small decode steps.*

Only numbers worth memorizing

- **512 + 256 tokens:** naive **163,712** vs needed **768** (≈ **213×**); cache ≈ **96 MB**.

Derive, don't memorize

1. Naive work is a staircase

Step k passes over P + k tokens; add the steps.

$$\sum_{k=0}^{N-1}(P+k) = NP + \frac{N(N-1)}{2} \qquad 512 \times 256 + \frac{256 \times 255}{2} = 131072 + 32640 = 163712$$

2. The waste ratio approaches the reply length ◆

$$\frac{NP + N^2/2}{P+N} \to N \ (P \gg N) \qquad 213 \ (N=256), \quad 461 \ (N=512), \quad 967 \ (N=1024)$$

INTUITION

◆ Each generated token reruns roughly the whole context, so N tokens cost about N full passes. That is why the waste is quadratic in total length.

3. What to keep, and what it costs

$$\text{cache} = \text{tokens} \times \text{KV/token} = 768 \times 128 \text{ KB} \approx 96 \text{ MB}$$

INTUITION

Causal mask ⇒ old K, V never change ⇒ store them once. Old queries are never reused, and old MLP outputs only fed the next layer's K, V.

Naive loop vs KV cache

	Naive loop	KV cache
Work per step	Whole sequence (P + k tokens)	One new token
Total work	NP + N(N − 1)/2	P + N
Time per token	Keeps climbing	Roughly flat ◆ (grows slowly with cache reads)
Extra memory	None	~128 KB per token
Outputs	Reference	Identical (exact)

Symptom → diagnosis

Symptom	Diagnosis
Time per token climbs as the reply grows	No KV cache: every step reruns the sequence
GPU memory grows with conversation length	Expected: the cache adds ~128 KB per token
◆ Cached and uncached outputs disagree beyond float noise	Cache bug (wrong positions or a stale entry)

Rapid-fire Q&A

Question	Crisp answer
Why is the naive loop wasteful?	It reruns every position each step and keeps only the last logits.
Why can old work be reused at all?	The causal mask: position i depends only on tokens 1 … i.
Why not cache queries or MLP outputs?	Old queries are never used again; MLP outputs only built the next layer's K, V.
Is the KV cache an approximation?	No: outputs are identical to the naive loop.
How does the waste scale?	Quadratically in length; the ratio is a bit under the reply length.

TRAP

- Calling the KV cache approximate: it's exact.
- Thinking the waste is linear: total naive work is quadratic in length.
- Forgetting the price: cache memory grows with every token of every request.

15 · Understand & Measure the Prefill Phase

Full notes

Say it in 30 seconds

> *Prefill runs the whole prompt through the model in one pass, writing every token's K and V into the cache in every layer, and the last position's logits give the first token, so TTFT is mostly prefill. Each weight byte is reused by every prompt token, so a few hundred tokens push arithmetic intensity past the ~300 ridge and prefill is compute-bound. Estimate it as 2 × params × tokens ÷ achievable FLOP/s: about 65 ms for 2k tokens on Llama 3 8B at half of H100 peak, plus an n^2 attention term for long prompts. Measure with single requests, max_tokens = 1, warm-up and medians over a length sweep, then fit a line.*

Only numbers worth memorizing

- **Llama 3 8B, 2k-token prompt on an H100:** ~**65 ms** (half of peak), ~**30k tokens/s**; attention adds ~**3%** at 2k, ~**+54%** at 32k.

Derive, don't memorize

1. Intensity of one matmul with T tokens

FLOPs: $2Th^2$. Bytes: the h^2 weights once, plus T input and T output activations (2 bytes each).

$$\text{AI} = \frac{2Th^2}{2h^2 + 4Th} = \frac{Th}{h + 2T} \qquad T = 1:1, \quad 500:\approx 400, \quad 2000:\approx 1000$$

INTUITION

For small T the weights dominate the bytes, so AI ≈ T: every extra token is nearly free math on bytes already read. ◆ For huge T it saturates near h/2, because activations dominate.

2. Prefill time from the FLOPs

$$t \approx \frac{2 \cdot N \cdot n}{\text{MFU} \times \text{peak}} = \frac{2 \times 8\text{B} \times 2000}{0.5 \times 989\ \text{TFLOPS}} \approx 65\ \text{ms} \quad \Rightarrow \quad \frac{2000}{65\ \text{ms}} \approx 30\text{k}\ \text{tok/s}$$

3. When attention starts to matter

Causal attention is about half of an n × n square per layer (scores plus value mixing), against 2N per token for the weights.

$$\frac{2Ln^2h}{2Nn} = \frac{Lhn}{N} = \frac{32 \times 4096 \times n}{8\text{B}} \approx 3\%\ (2\text{k}), \quad 54\%\ (32\text{k}), \quad 100\%\ (\approx 61\text{k})$$

4. Read the sweep as a line

$$\text{TTFT}(n) \approx t_0 + \frac{n}{\text{prefill tok/s}} \qquad \text{intercept} = t_0, \quad \frac{1}{\text{slope}} = \text{throughput}$$

What the sweep tells you

Feature	Meaning
Intercept	Fixed cost: networking, tokenization, scheduling
Slope	Time per prompt token
1 / slope	Prefill throughput (tokens/s)
◆ Upward curve at long lengths	The n^2 attention term
High GPU util and power	Compute-bound work, as expected

Symptom → diagnosis

Symptom	Diagnosis
Measured throughput far below half the napkin	Something's wrong: investigate the setup
TTFT numbers jump around	Concurrent requests, no warm-up, or no median
Other users' streams stall during long prompts	A big prefill occupies the GPU (fix: chunked prefill)
The same long system prompt is slow every time	Shared prefix recomputed (fix: prefix caching)

Rapid-fire Q&A

Question	Crisp answer
What two jobs does prefill do?	Fills the KV cache and produces the first token.
Why can prefill run all tokens at once?	The whole prompt is known up front; no token waits for another.
Why is prefill compute-bound?	Each weight byte is reused by every prompt token.
Why set max_tokens = 1 when measuring?	The response time is then basically TTFT.
When does attention dominate prefill?	Around 60k tokens on Llama 3 8B; ~3% at 2k.

TRAP

- Measuring TTFT under concurrent load: other requests contaminate it.
- Estimating at peak FLOP/s: real kernels reach about half.
- Assuming prefill is linear forever: attention bends it upward at long prompts.

16 · Understand & Measure the Decode Phase

Full notes

Say it in 30 seconds

> *Decode runs one token per request through every layer as vector-times-matrix, rereading all weights plus that request's KV cache at about one op per byte, so it's memory-bound and time per token is at least bytes over bandwidth: about 5 ms for Llama 3 8B at 4k context on an H100, or 200 tokens/s, and real batch-1 runs are 3–4× slower. Batching shares the weight read, so batch 64 gives 15 ms per step but over 4,000 tokens/s, until KV memory runs out. Measure with a short prompt and forced long output, compute TPOT and the inter-token gap tail, sweep concurrency, and report achieved bandwidth, not GPU utilization.*

Only numbers worth memorizing

- **Llama 3 8B, H100, 4k context:** floor ≈ **5 ms/token** (≤ 200 tok/s); batch 16 ≈ **7.4 ms, 2.2k tok/s**; batch 64 ≈ **15 ms, 4.3k tok/s**, 34 GB of KV.

Derive, don't memorize

1. Decode vs prefill, per token

$$\frac{60\ \text{ms}}{2000\ \text{tokens}} = 0.03\ \text{ms} \quad \text{vs} \quad \approx 5\ \text{ms} \quad \Rightarrow \quad 100\times\ \text{or more}$$

2. The floor includes this request's cache

$$t \geq \frac{W + C \cdot \text{KV}}{\text{BW}} = \frac{16.06\ \text{GB} + 4096 \times 128\ \text{KB}}{3.35\ \text{TB/s}} = \frac{16.6\ \text{GB}}{3.35\ \text{TB/s}} \approx 5.0\ \text{ms} \qquad C = 32\text{k}: \approx 6.1\ \text{ms}$$

> **INTUITION**
>
> The weights dominate until the cache rivals them: ◆ at batch 1 that takes ~122k tokens (05). Real kernels add small-kernel and launch costs, landing 3–4× above.

3. Batching: throughput = batch ÷ step time

$$\frac{B}{t(B)}: \quad \frac{16}{7.4\ \text{ms}} \approx 2170\ \text{tok/s} \qquad \frac{64}{15.1\ \text{ms}} \approx 4250\ \text{tok/s} \qquad \text{KV} = 64 \times 0.54\ \text{GB} \approx 34\ \text{GB}$$

> **INTUITION**
>
> Per user, tokens/s = 1 ÷ t(B): 200 → 136 → 66. Total goes up 21× while each user slows ~3×.

Batch size vs speed (floor model, 4k context)

Batch	Step time	Per user	GPU total	KV in memory
1	5.0 ms	202 tok/s	202 tok/s	0.5 GB
16	7.4 ms	136 tok/s	2,174 tok/s	8.6 GB
64	15.1 ms	66 tok/s	4,252 tok/s	34 GB

Measurement recipe

Do	Why
Short prompt, long output	Prefill barely matters
Force the output length (ignore EOS)	Every run generates the same count
TPOT = (E2E − TTFT) ÷ (N_out − 1)	Decode pace without the first-token pause
Keep every inter-token gap	The tail shows stalls the mean hides
Sweep concurrency 1 → 64	Traces the latency vs throughput curve
Report achieved bandwidth	Bytes per step ÷ step time: distance to the floor

Symptom → diagnosis

Symptom	Diagnosis
GPU util ~100% but decode is slow	Util only means a kernel ran; check achieved bandwidth
Batch-1 TPOT 3–4× the floor	Small kernels and launch overheads
Mean TPOT fine, users see stutters	Inter-token gap tail: stalls
Out of memory as concurrency rises	KV cache, not math, caps the batch

Rapid-fire Q&A

Question	Crisp answer
Why is decode memory-bound?	One token per step reuses each weight byte once: ~1 op/byte vs ~300 needed.
What two things does every step read?	All the weights and the request's whole KV cache.
How does batching help?	The weight read is shared by every request in the batch.
What caps the batch?	KV memory: each request keeps its cache on the GPU.
How close to the floor am I?	Achieved bandwidth ÷ peak, e.g. 16.6 GB ÷ 12 ms ≈ 1.4 TB/s ≈ 41%.

TRAP

- Treating the floor as the expected speed: it's the ceiling.
- Trusting GPU utilization during decode: the math units can be idle.
- Averaging TPOT only: stalls hide in the gap tail.

17 · Build a KV Cache

Full notes

Say it in 30 seconds

> *Per layer I keep a key and a value tensor shaped [batch, KV heads, max length, head dim], plus a position pointer. That's 128 KiB per token for Llama 3 8B, 1 GiB for an 8k reservation. I preallocate and write in place instead of concatenating, which avoids quadratic copying and keeps shapes static for CUDA graphs. Prefill fills slots 0 to P−1 at once under a causal mask; each decode step writes at the pointer, reads only up to it, and advances. The classic bugs are RoPE at the wrong position, attending to zero-filled slots ($e^0 = 1$), not resetting between requests, and copying KV heads instead of broadcasting. I test that greedy outputs match the uncached loop, bytes match the formula, and step time stays flat.*

Only numbers worth memorizing

- **Llama 3 8B cache:** per layer K, V = [B, **8**, L_max, **128**]; **8,192 tokens = 1 GiB** per request.

Derive, don't memorize

1. Bytes the cache should allocate (test 2)

$$\text{bytes} = 2 \cdot L \cdot H_{\text{kv}} \cdot d \cdot L_{\max} \cdot b \cdot B = 128 \text{ KiB} \times 8192 = 1 \text{ GiB per request}$$

2. Why concatenation is quadratic

Each step copies everything so far, then adds one.

$$\sum_{t=1}^{L} t = \frac{L(L+1)}{2} = \frac{8192 \times 8193}{2} \approx 33.6\text{M copies} \quad \text{vs} \quad 8192 \text{ writes}$$

3. Reserved but empty

$$\text{waste} = R \cdot (L_{\max} - \bar{L}) \cdot 128 \text{ KiB} = 8 \times (8192 - 1000) \times 128 \text{ KiB} \approx 7 \text{ GiB}$$

4. What an unmasked empty slot steals ◆

Each zero slot scores 0 and contributes $e^0 = 1$ to the softmax denominator.

$$w_{\text{empty}} = \frac{n_{\text{empty}}}{\sum_i e^{s_i} + n_{\text{empty}}}$$

> **INTUITION**
>
> ◆ Early in a long reservation almost every slot is empty, so the weight they steal can be huge. Masking to $-\infty$ (or slicing to the pointer) makes it exactly 0.

The four bugs

Bug	Symptom	Fix
RoPE at position 0 in decode	First token fine, then gibberish	Rotate by the pointer's position
Reading empty slots	Attention diluted toward zeros	Slice or mask to the filled length
Not resetting	Next user attends to the previous user's text	Reset the pointer per request
Copying KV heads to 32	◆ 4× the memory (512 KiB/token)	Store 8, broadcast at attention

Three tests

Test	Pass criterion
Correctness	100 greedy tokens identical with/without cache; logits within tolerance (16-bit)
Memory	Allocated bytes = 2 · layers · KV heads · head dim · length · bytes
Speed	Per-step time nearly flat with the cache, climbing without

Rapid-fire Q&A

Question	Crisp answer
Why preallocate instead of torch.cat?	cat copies everything every step (quadratic); preallocation is one write and fixed shapes.
Why do fixed shapes matter?	CUDA graphs and compilers need static shapes.
What does the pointer do?	Marks the filled length: where to write next and how far to read.
Why aren't cached logits bit-identical?	16-bit kernels compute in a different order; they agree within tolerance.
What's the cost of preallocating?	Reserved-but-unused memory caps how many requests fit (paged memory fixes it).

TRAP

- Feeding the decode token at position 0: RoPE needs its true position.
- Assuming zero-filled slots are harmless: $e^0 = 1$ gives them weight.
- Expecting bit-identical logits in 16-bit: compare tokens exactly, logits with a tolerance.

18 · Calculate Model & Runtime Memory

Full notes

Say it in 30 seconds

> *GPU memory is weights plus runtime overhead (CUDA context, NCCL, workspaces) plus activations (set by tokens per forward pass) plus the KV cache plus headroom for CUDA graphs and fragmentation. Only the cache serves users, so measure the overhead and give it the rest. vLLM takes 90% of the GPU, subtracts measured weights and a profile run's peak, and allocates the remainder as cache: for Llama 3 8B on an H100, 72 − 16 − 3 ≈ 53 GB, about 400k tokens. Llama 3 70B needs 141 GB: tensor parallel 2 leaves no cache, TP 4 leaves about 135 GB, roughly 410k tokens at 320 KiB each.*

Only numbers worth memorizing

- **Llama 3 8B on one H100:** 72 GB budget − 16 − ~3 ≈ **53 GB of cache ≈ 400k tokens**. **70B:** 141 GB, needs **TP 4** for useful cache.

Derive, don't memorize

1. The engine's cache budget

$$\mathrm{KV} = u \cdot \mathrm{HBM} - W - A_{\text{peak}} = 0.9 \times 80 - 16 - 3 \approx 53\ \mathrm{GB} \quad \Rightarrow \quad \frac{53\ \mathrm{GB}}{128\ \mathrm{KiB}} \approx 400\mathrm{k}\ \text{tokens}$$

2. Activation spikes scale with tokens per step

$\text{gate} + \text{up} = T \cdot 2i \cdot 2\ \mathrm{B} = 8192 \times 2 \times 14336 \times 2 \approx 0.47\ \mathrm{GB}$ $\quad$ $\text{logits} = S \cdot V \cdot 4\ \mathrm{B} = 256 \times 128256 \times 4 \approx 131\ \mathrm{MB}$

> **INTUITION**
>
> These are why the profile run uses the **largest batch it will ever allow**: the peak, not the average, must fit.

3. Tensor parallel: split the weights, then check what's left

$$\mathrm{KV}_{\text{total}} = \mathrm{TP} \cdot \left(72 - \frac{141}{\mathrm{TP}} - 3\right): \quad \mathrm{TP} = 2 \Rightarrow < 0, \quad \mathrm{TP} = 4 \Rightarrow 4 \times 33.7 \approx 135\ \mathrm{GB}$$

4. 70B's KV per token, and the token count

$$2 \cdot 80 \cdot 8 \cdot 128 \cdot 2\ \mathrm{B} = 320\ \mathrm{KiB} \qquad \frac{135\ \mathrm{GB}}{320\ \mathrm{KiB}} \approx 410\mathrm{k}\ \text{tokens}$$

> **INTUITION**
>
> ◆ 70B has 80 layers vs 32 but the same 8 KV heads, so 2.5× the KV per token. Four GPUs bought 2.5× the cache memory (135 vs 53 GB), which the bigger tokens eat: about the same ~400k tokens.

The memory budget, piece by piece

Piece	Scales with	Llama 3 8B / H100
Weights	Params × bytes	16 GB (BF16)
Runtime	Process, GPU count (NCCL)	A few hundred MB + buffers
Activations	Tokens per forward pass	~3 GB peak (profiled)
KV cache	Whatever is left	~53 GB ≈ 400k tokens
Headroom	10% outside the budget	8 GB: graphs (1–5 GB), fragmentation

Symptom → diagnosis

Symptom	Diagnosis
Crash while loading	Weights don't fit: more GPUs or lower precision
Crash while sizing the cache	Budget too small for the max length: lower it or raise the budget
Crash during graph capture	Not enough headroom outside the budget
Tool says 15 for "16 GB"	It prints GiB

Rapid-fire Q&A

Question	Crisp answer
Why isn't 80 − 16 = 64 GB free?	Runtime, activations and headroom claim their share first.
Where do you get the exact weight size?	`total_size` in model.safetensors.index.json.
What do activations depend on?	Tokens in one forward pass, not model size alone.
What does vLLM's 90% mean?	Its memory budget; the last 10% is left for CUDA graphs and surprises.
Why does 70B on 2 GPUs fail to serve?	70.6 GB per GPU leaves ~nothing of 72 GB for overhead and cache.

TRAP

- Subtracting only the weights: runtime, activations and headroom are real.
- Mixing GB and GiB: 16 GB is ~15 GiB.
- Stopping at "the weights fit": with no room for cache, it can't serve.

19 · Calculate KV Memory, Context & Concurrency Limits

Full notes

Say it in 30 seconds

> *Bytes per token is 2 × layers × KV heads × head dim × bytes: 128 KiB for Llama 3 8B, 512 for Llama 2 7B without GQA, 320 for 70B, ~69 for DeepSeek-V3 with MLA. A request holds that times prompt plus output, so 8k tokens is 1 GiB and 128k is 16 GiB. Concurrency is the KV pool divided by per-request cache: an H100's ~53 GB, about 400k tokens, fits 197 requests at 2k, 49 at 8k, 3 at 128k. Context and concurrency trade one for one, so the workload sets the bill: 100 chat users fit but only 31 document users. GQA or MLA, an FP8 cache, more GPUs, and paged allocation by actual length move the curve.*

Only numbers worth memorizing

- **H100, Llama 3 8B, ~400k-token pool: 197 / 49 / 12 / 3** requests at 2k / 8k / 32k / 128k.

Derive, don't memorize

1. Concurrency is one division

$$\text{concurrency} = \frac{\text{pool}}{\text{bytes/token} \times \text{length}} = \frac{53\ \text{GB}}{128\ \text{KiB} \times 8192} \approx \frac{404\text{k}}{8192} \approx 49$$

2. Plan a service: users × length × bytes/token vs the pool

$$100 \times 1500 \times 128\ \text{KiB} \approx 20\ \text{GB}\ (\text{fits}) \qquad 100 \times 13000 \times 128\ \text{KiB} \approx 170\ \text{GB} \Rightarrow \frac{404\text{k}}{13000} \approx 31$$

3. The levers multiply

Concurrency ∝ pool ÷ (bytes per token × allocated length), so each lever is a factor.

$$\text{GQA}\ 4\times\ (12 \to 49) \qquad \text{FP8 KV}\ 2\times\ (\to 98) \qquad \text{actual length}\ \frac{8192}{1500} \approx 5.5\times\ (\to 270)$$

> **INTUITION**
>
> ◆ Factors stack: FP8 cache plus paged allocation ≈ 2 × 270 ≈ 540 requests on the same GPU.

4. Why DeepSeek-V3 is so small ◆

MLA stores one 512-number latent plus a 64-number RoPE key per layer, instead of K and V for every head.

$$(512 + 64) \times 61\ \text{layers} \times 2\ \text{B} \approx 68.6\ \text{KiB/token}$$

KV cache per token, by architecture

Model	Why	Per token	8k request
Llama 2 7B	32 layers, 32 KV heads (MHA)	512 KiB	4 GiB
Llama 3 8B	32 layers, 8 KV heads (GQA)	128 KiB	1 GiB
Llama 3 70B	80 layers, 8 KV heads	320 KiB	2.5 GiB
DeepSeek-V3 671B	MLA latent, 61 layers	~69 KiB	~0.54 GiB

Symptom → diagnosis

Symptom	Diagnosis
Far fewer concurrent users than expected	Long contexts: each fills many users' seats
Same GPU fine for chat, overloaded for RAG	Per-request cache is ~9× bigger (13k vs 1.5k tokens)
Startup "max concurrency" is low	Max length × bytes/token too big for the pool: check by hand
Memory reserved but mostly empty	Allocating max length per request (fix: paged KV)

Rapid-fire Q&A

Question	Crisp answer
What sets bytes per token?	The architecture: layers, KV heads, head dim, cache precision.
How does context trade with concurrency?	One for one: double the length, halve the requests.
Why is a huge context window expensive?	One user filling it takes many users' seats.
How does GQA help serving?	8 KV heads instead of 32: 4× more tokens in the same pool.
What does FP8 KV cost?	Doubles the pool, at a small risk to quality.

TRAP

- Sizing for the average model, not the workload: RAG and chat differ ~9× per request.
- Reserving max length per request: real conversations rarely reach it.
- Forgetting output tokens: the cache holds prompt plus everything generated.

20 · TTFT, TPOT, ITL, E2E & Throughput

Full notes

Say it in 30 seconds

> *Every serving metric is part of one request's timeline. TTFT is send to first token, which at the client includes network, queueing and prefill, and needs streaming to measure. ITL is each gap between tokens; TPOT is one request's average gap, end-to-end minus TTFT over output tokens minus one, and I'd also keep the max gap since a stall barely moves the mean. E2E is roughly TTFT plus TPOT times N minus one, so it's dominated by output length; compare like with like. When aggregating, say whether you average per request or per token: the same data gave 30 vs 10.4 ms. Throughput is 1 over TPOT per user, summed for the system, and you must check whether it counts input tokens or requests, which clock it uses, and whether it timed messages or tokens.*

Only numbers worth memorizing

- **20 ms/token = 50 tok/s** (faster than reading). **Same data, two averages:** 30 vs 10.4 ms.

Derive, don't memorize

1. Per-request vs per-token average

$$\frac{50+10}{2} = 30 \text{ ms} \qquad \frac{9\times 50 + 999\times 10}{9+999} = \frac{10440}{1008} \approx 10.4 \text{ ms}$$

> **INTUITION**
>
> The per-token average weights each request by its gap count, so long replies dominate it. ◆ A few short, slow replies almost disappear from it.

2. Stalls hide in the mean ◆

$$\frac{298\times 20 + 500}{299} \approx 21.6 \text{ ms} \quad (+8\%) \qquad \max = 500 \text{ ms } (25\times)$$

3. Throughput per user, per system, and inflated ◆

$$\frac{1}{25 \text{ ms}} = 40 \qquad 64 \times 40 = 2560 \text{ out tok/s} \qquad \times \frac{1000+300}{300} \approx 11\text{k total tok/s} \qquad \frac{2560}{300} \approx 8.5 \text{ req/s}$$

4. Chunking multiplies apparent TPOT by tokens per message

$$\text{message gap} = k \times \text{TPOT}: \quad 4 \times 20 \text{ ms} = 80 \text{ ms}$$

Which metric answers what

Metric	Spans	Driven by	Tells you
TTFT	Send → first token	Network, queue, prompt length (prefill)	Does the app feel responsive?
ITL	One gap	Each decode step	Raw material for TPOT and stalls
TPOT	Avg gap, one request	Model size, batch, context	Reading speed (1 ÷ TPOT tok/s)
Max gap	Worst gap, one request	Stalls, preemption ◆, long prefills sharing the batch ◆	Did the text freeze?
E2E	Send → last token	Output length, mostly	Total wait, only comparable at similar lengths
Throughput	Tokens per second	Concurrency	Capacity (per user or system)

Symptom → diagnosis

Symptom	Diagnosis
Two tools report different TPOT on one run	One averages per request, the other per token
TPOT several times worse than expected	Timing network messages, not tokens (chunking, speculative decoding)
TTFT equals E2E	Not streaming: the first token came with the last
Benchmark throughput far above decode rate	Input tokens counted in the headline
Client TTFT much higher than server metrics	Network plus your own app's queue
One request "slow" on E2E alone	It wrote a longer reply; compare TTFT and TPOT

Rapid-fire Q&A

Question	Crisp answer
Why N − 1 in TPOT?	The first token's wait is already counted in TTFT.
Why keep the max gap?	One 500 ms stall barely moves the mean but freezes the text.
Which TPOT average is right?	Either; state it. Per request treats users equally, per token treats tokens equally.
Client or server clock?	Client for what users feel; server to diagnose the engine.
Does more concurrency help throughput?	System throughput up, each user's TPOT a little worse.

TRAP

- Quoting E2E without the output length.
- Reporting "tokens/s" without saying output, total, or per user.
- Averaging TPOT without saying per request or per token.

21 · Concurrency, Request Rate & Arrival Patterns

Full notes

Say it in 30 seconds

> *Load shape changes every number. A closed loop fixes concurrency: users wait for replies, so a slow server just gets fewer requests and queues stay hidden. An open loop fixes the arrival rate like real users, so queues show. Little's law ties them: in flight equals rate times time in system, 4 per second at 6 seconds is 24. At the same average rate, burstier arrivals give far worse tails: on a server 80% busy, P99 wait went from 0 to 0.9 to 2.7 seconds for uniform, Poisson and bursty. And time in system scales like 1 over 1 minus utilization, 5× at 80% and 20× at 95%, so sweep concurrency for max throughput, sweep Poisson rate for latency targets, add bursts, and report the achieved rate.*

Only numbers worth memorizing

- **Knee:** 1 ÷ (1 − ρ) = **2× / 5× / 10× / 20×** at 50 / 80 / 90 / 95% busy.

Derive, don't memorize

Let λ = arrival rate, W = time in system, S = service time, ρ = utilization.

1. Little's law: in flight = rate × time in system

Every request occupies a slot for W seconds and λ new ones show up each second, so λW are inside at any moment.

$$L = \lambda W = 4 \times 6\ \text{s} = 24 \qquad \lambda = \frac{L}{W} = \frac{32}{6\ \text{s}} \approx 5.3\ \text{req/s}$$

> **INTUITION**
>
> Converts a concurrency sweep into a rate and back. ◆ If W rises with load, the same concurrency serves fewer req/s.

2. Utilization = rate × service time

$$\rho = \lambda S = 10\ \text{req/s} \times 80\ \text{ms} = 0.8$$

3. The knee: time in system = S ÷ (1 − ρ)

The busier the server, the likelier a new request finds someone ahead of it, and the line feeds on itself.

$$\frac{W}{S} = \frac{1}{1-\rho}: \quad 0.5 \Rightarrow 2 \quad 0.8 \Rightarrow 5 \quad 0.9 \Rightarrow 10 \quad 0.95 \Rightarrow 20$$

> **INTUITION**
>
> Halving the idle time (10% → 5%) doubles the time in system. ◆ At 99% busy it's 100×.

4. Poisson wait with a fixed service time ◆

Random arrivals into a server with a constant 80 ms per request (an M/D/1 queue) wait on average:

$$W_q = \frac{\rho}{2(1-\rho)} \cdot S = \frac{0.8}{0.4} \times 80 \text{ ms} = 160 \text{ ms}$$

INTUITION

◆ Matches the lesson's simulated 166 ms. Random service times (M/M/1) double it to 320 ms; burstier arrivals push it higher still.

Closed vs open loop

	Closed loop	Open loop
You fix	Concurrency (e.g. 8 users)	Arrival rate (e.g. 10 req/s)
Server slows down	Fewer requests sent: self-throttles	Same arrivals: queue grows
Hides	Queueing delay	Nothing: shows it
Use for	Max throughput (sweep 1, 2, 4, …)	Latency targets (Poisson, then bursts)

Symptom → diagnosis

Symptom	Diagnosis
Two teams, same server: great TTFT vs terrible tails	Different load shapes (closed vs open, uniform vs bursty)
Benchmark latency fine, production TTFT bad	Closed-loop test hid the queue; prod is open and bursty
Achieved rate below configured rate	Server couldn't keep up: past capacity
Latency fine at 70%, explodes at 90%	The knee: 1 ÷ (1 − ρ)
Mean wait OK, P99 awful at the same load	Bursty arrivals
LLM server fine, then sudden latency cliff	Batch or KV cache full: batching stopped stretching capacity

Rapid-fire Q&A

Question	Crisp answer
Which loop matches real users?	Open: they don't wait for each other.
Little's law?	L = λW (derivation 1).
Why not run GPUs at 99% busy?	Time in system ≈ 100× service ◆ (derivation 3).
How do tools model bursts?	Gamma arrivals with burstiness below 1 (Poisson = 1 ◆).
What to report besides config?	The achieved request rate.

TRAP

- Quoting latency without the load shape (concurrency or rate, and arrival pattern).
- Using only closed-loop tests to set latency SLOs.
- Treating high utilization as efficiency.

22 · P50, P95, P99 & Variance

Full notes

Say it in 30 seconds

> *Before quoting a P99 I'd say which metric and which population: TTFT, TPOT, gaps and E2E have different tails, per-token gap percentiles are dominated by long replies, and a blend of short and long prompts moves its P99 whenever the mix shifts, so segment first. Spread comes from queueing, lengths, batch interference, preemption, cache hits and the hardware. Measure from the intended send time on a scheduled load, or coordinated omission hides stalls: 50 ms measured vs 1.5 s true. Never average percentiles; merge histograms. And repeat runs: medians move under 1%, tails much more.*

Only numbers worth memorizing

- **Coordinated omission:** P99 **50 ms** measured vs **≈ 1.5 s** true (2 s stall every 50 s). **Averaged P99s:** 1.3 s vs true **2.7 s**.

Derive, don't memorize

1. Coordinated omission: count what should have been sent ◆

At 10 req/s, a 2 s stall should hold 20 requests. Per 50 s that's 500 requests: 20 slow (4%) on schedule, but only 1 slow (0.2%) for a waiting client.

$$\frac{20}{500} = 4\% > 1\% \Rightarrow \text{P99 in the stall} \qquad \frac{1}{500} = 0.2\% < 1\% \Rightarrow \text{P99} = 50 \text{ ms}$$

> **INTUITION**
>
> ◆ The 20 slow ones waited 0–2 s, evenly spread. The slowest 1% of all = the slowest quarter of them: ≥ 1.5 s.

2. Why P99s don't average ◆

The hourly P99 is the slowest 1% of **all** samples. If the incident window holds 1/12 of them and nearly all the slow ones:

$$\text{hourly P99} \approx \text{P}(100 - 12 \times 1) = \text{P88 of the incident window} \qquad \text{average} = \frac{11 \times 1.0 + 4.6}{12} \approx 1.3 \text{ s}$$

> **INTUITION**
>
> The average dilutes one terrible window with eleven fine ones; the true percentile is decided by it.

3. A blend's P99 lives in the slow group ◆

If short prompts never get that slow, the blend's slowest 1% all come from the long share s:

$$\text{P99}_{\text{blend}} = \text{P}(100 - \frac{1}{s}) \text{ of the long group}: \quad s = 0.4 \Rightarrow \text{P97.5 of long documents}$$

Sources of variance → what to check

Source	Shows up in	Check
Queueing	TTFT tail	Arrival bursts, load vs capacity (21)
Lengths	TTFT (prompt), E2E (reply)	Segment by length
Batch interference	Inter-token gap tail	Long prefills sharing steps with decode
Preemption	Huge gaps, E2E outliers	KV cache usage near full
Cache hits	Fast TTFT hump	Segment by cached vs not
Clocks, network	Everything, drifting	GPU temperature and power; client vs server timing

Symptom → diagnosis

Symptom	Diagnosis
Tail looks great, users report freezes	Coordinated omission: client waited during stalls
P99 jumped but no component got slower	Traffic mix shifted: segment first
Dashboard P99 calm during a known incident	Averaging per-window percentiles
Two humps in the latency histogram	Two populations (short vs long, cached vs not)
Timeouts vanish from results	Counted as missing, not as slow

Rapid-fire Q&A

Question	Crisp answer
First question about any P99?	Of which metric, over which population?
How to avoid coordinated omission?	Scheduled load, time from intended send (derivation 1).
How to get P99 across machines?	Merge histograms, then take the percentile.
Medians vs tails across reruns?	Medians under 1%; tails wander, so report spread.

TRAP

- Measuring latency with a client that waits for each reply.
- Averaging per-window or per-machine percentiles.
- Comparing P99s across different traffic mixes.